ARRAY × LAMBDA

GAME PROGRAMMING COURSE — C# BASICS

配列操作とラムダ式

最大・合計・ソートから、1行で書けるラムダ式まで

対象：1年生（クラスまで既習）

LESSON 1

1

配列操作の基本

目標：C# に用意された機能で、最大・合計・ソート・検索ができる

復習

配列：同じ型の値をまとめた箱の列

C# Program.cs

```
int[] scores = { 70, 45, 90, 60, 85 };
Console.WriteLine(scores.Length);
Console.WriteLine(scores[0]);
for (int i = 0; i < scores.Length; i++)
{
    Console.WriteLine(scores[i]);
}
```

出力

```
5
70
70
45
90
60
85
```



POINT 番号は 0 から、要素数は Length。ここまでは前回までと同じ。今日は「配列をまとめて操作する機能」を覚える。

今日のテーマ

for で書いていた処理が、1 行になる

これまで：for で自分で書く（最大値）

C# Program.cs

```
1  int max = scores[0];
2  for (int i = 1; i < scores.Length; i++)
3  {
4      if (scores[i] > max)
5      {
6          max = scores[i];
7      }
8  }
```

今日：用意された機能を使う

C# Program.cs

```
1  int max = scores.Max();
```

- 結果は同じ（90）
- 短いので**間違えにくい**
- ただし、中で何をしているかは for 版と同じ。**for 版も書けることが前提**

POINT Max や Sum などは using System.Linq; で使えるようになる機能。Paiza の書き始めで毎回書いている 3 行目がそれ。

まずは全体

配列操作をまとめて試すプログラム

C# Program.cs

```
1  string text = Console.ReadLine();
2  int[] scores = text.Split(" ").Select(int.Parse).ToArray();
3  Console.WriteLine(scores.Max());
4  Console.WriteLine(scores.Min());
5  Console.WriteLine(scores.Sum());
6  Console.WriteLine(scores.Average());
7  Console.WriteLine(Array.IndexOf(scores, 90));
8  Console.WriteLine(scores.Contains(60));
9  Array.Sort(scores);
10 Console.WriteLine(string.Join(" ", scores));
11 Array.Reverse(scores);
12 Console.WriteLine(string.Join(" ", scores));
```

入力例

```
70 45 90 60 85
```

出力

```
90
45
350
70
2
True
45 60 70 85 90
90 85 70 60 45
```

POINT このあと「3〜6 行目」「7〜8 行目」「9〜12 行目」に分けて見ていく。

3〜6 行目

最大・最小・合計・平均

C# Program.cs

```
1  Console.WriteLine(scores.Max());
2  Console.WriteLine(scores.Min());
3  Console.WriteLine(scores.Sum());
4  Console.WriteLine(scores.Average());
```

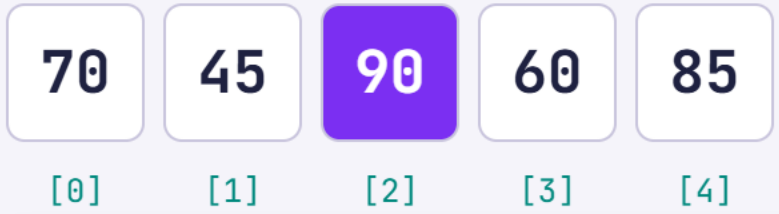
書き方	意味	結果
scores.Max()	一番大きい値	90
scores.Min()	一番小さい値	45
scores.Sum()	合計	350
scores.Average()	平均（double）	70

機能	for で書くと（if / for 授業の型）
Max() / Min()	型⑤ 最大・最小：max = 最初の値 → 大きければ入れかえ
Sum()	型④ 合計する：total = 0 → total += 値
Average()	型④ の合計 ÷ 要素数（小数にするため double にする）

注意 Average() は小数（double）を返す。int に入れるとエラー：double average = scores.Average();

7〜8 行目

探す：Array.IndexOf と Contains



C# Program.cs

```
1  Console.WriteLine(Array.IndexOf(scores, 90));
2  Console.WriteLine(Array.IndexOf(scores, 100));
3  Console.WriteLine(scores.Contains(60));
```

出力

```
2
-1
True
```

書き方	返すもの
Array.IndexOf(配列, 値)	最初に見つかった番号 なければ -1
配列.Contains(値)	含まれていれば True

- 「何番目」は IndexOf + 1
- Contains はそのまま if の条件に使える

9〜12 行目

並べ替え：Array.Sort と Array.Reverse

元の配列

70	45	90	60	85

Array.Sort(scores); のあと（小さい順・昇順）

45	60	70	85	90

さらに Array.Reverse(scores); のあと（大きい順・降順）

90	85	70	60	45

POINT

Array.Sort は**元の配列そのものを並べ替える**（新しい配列は作らない）。大きい順にしたいときは Sort → Reverse の 2 行。

10・12 行目

配列をまとめて出力する：string.Join

for で 1 つずつ出力（1 行に 1 つ）

C# Program.cs

```
for (int i = 0; i < scores.Length; i++)
{
    Console.WriteLine(scores[i]);
}
```

出力

```
45
60
70
```

string.Join で 1 行に（スペース区切り）

C# Program.cs

```
Console.WriteLine(string.Join(" ", scores));
```

出力

```
45 60 70
```

POINT Paiza の出力が「1 行に 1 つ」なら for、「スペース区切りで 1 行」なら string.Join。出力例の形をよく見て選ぼう。

よくある間違い

配列操作でつまずきやすいポイント

✕ 間違い	何が起きる？	○ 正しい書き方
int[] sorted = Array.Sort(scores);	エラー（Sort は何も返さない）	Array.Sort(scores); だけ書く
int average = scores.Average();	エラー（Average は double）	double average = …
Console.WriteLine(scores);	System.Int32[] と表示される	string.Join(" ", scores)
Max() が使えない	using System.Linq; を書き忘れ	using を 3 行とも書く
ソート後に「元の何番目」を探す	Sort で順番が変わっている	探してから Sort する

注意 Array.Sort / Array.Reverse は「配列を書き換える」。Max / Sum などは「配列はそのまま、結果だけ返す」。

一覧

配列操作 一覧（演習中に見返そう）

書き方	意味	例（scores = {70, 45, 90}）→ 結果
scores.Length	要素数	3
scores.Max() / Min()	最大 / 最小	90 / 45
scores.Sum()	合計	205
scores.Average()	平均（double）	68.333…
Array.IndexOf(scores, 値)	何番目にあるか（なければ -1）	IndexOf(scores, 90) → 2
scores.Contains(値)	含まれているか	Contains(45) → True
Array.Sort(scores)	小さい順に並べ替え（配列が変わる）	{45, 70, 90}
Array.Reverse(scores)	順番を逆にする（配列が変わる）	{90, 45, 70}
string.Join(" ", scores)	スペース区切りの 1 つの文字列に	"70 45 90"

LESSON 2

2

ラムダ式

目標：ラムダ式で「条件つきの数える・選ぶ・変換する・並べ替える」が書ける

ラムダ式とは

名前のない、その場で使う小さな関数

ふつうの関数

C# Program.cs

```
1  static int AddBonus(int score)
2  {
3      return score + 5;
4  }
```

同じ意味のラムダ式

C# Program.cs

```
1  score ⇒ score + 5
```



受け取る値（引数）　「〜を返す」　返す値

POINT　=> は「〜を受け取って、〜を返す」と読む。score => score + 5 は「score を受け取って score + 5 を返す」。

まずは全体

ラムダ式を使った配列操作のプログラム

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  int passCount = scores.Count(score ⇒ score ⩾ 60);
3  int[] passScores = scores.Where(score ⇒ score ⩾ 60).ToArray();
4  int[] bonusScores = scores.Select(score ⇒ score + 5).ToArray();
5  bool hasZero = scores.Any(score ⇒ score == 0);
6  int[] ranking = scores.OrderByDescending(score ⇒ score).ToArray();
7  Console.WriteLine(passCount);
8  Console.WriteLine(string.Join(" ", passScores));
9  Console.WriteLine(string.Join(" ", bonusScores));
10 Console.WriteLine(hasZero);
11 Console.WriteLine(string.Join(" ", ranking));
```

出力

```
4
70 90 60 85
75 50 95 65 90
False
90 85 70 60 45
```

POINT 2〜6 行目がラムダ式。1 行ずつ、if / for 授業の「型」と見比べていく。

2 行目　Count

Count：条件に合うものを数える（型③）

for で書くと

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  int passCount = 0;
3  for (int i = 0; i < scores.Length; i++)
4  {
5      if (scores[i] ⩾ 60)
6      {
7          passCount++;
8      }
9  }
10 Console.WriteLine(passCount);
```

ラムダ式で書くと

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  int passCount = scores.Count(
3      score ⇒ score ⩾ 60);
4  Console.WriteLine(passCount);
```

出力

```
4
```

POINT

( ) の中は「数える条件」。score => score >= 60 は「点数を受け取って、合格なら true を返す」。true になった数を数えてくれる。

3 行目　Where

Where：条件に合うものだけ選ぶ（型②）

for で書くと

C# Program.cs

```
int[] scores = { 70, 45, 90, 60, 85 };
int count = 0;
for (int i = 0; i < scores.Length; i++)
{
    if (scores[i] ⩾ 60) { count++; }
}
int[] passScores = new int[count];
int index = 0;
for (int i = 0; i < scores.Length; i++)
{
    if (scores[i] ⩾ 60)
    {
        passScores[index] = scores[i];
        index++;
    }
}
Console.WriteLine(string.Join(" ", passScores));
```

ラムダ式で書くと

C# Program.cs

```
int[] scores = { 70, 45, 90, 60, 85 };
int[] passScores = scores
    .Where(score ⇒ score ⩾ 60)
    .ToArray();
Console.WriteLine(string.Join(" ", passScores));
```

出力

```
70 90 60 85
```

POINT　for だと「数えてから入れる」の 2 回ループが必要。Where なら 1 行。最後の ToArray() で配列に戻す。

4 行目　Select

Select：全部に同じ処理をした配列を作る（型①）

for で書くと

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  int[] bonusScores = new int[scores.Length];
3  for (int i = 0; i < scores.Length; i++)
4  {
5      bonusScores[i] = scores[i] + 5;
6  }
7  Console.WriteLine(string.Join(" ", bonusScores));
```

ラムダ式で書くと

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  int[] bonusScores = scores
3      .Select(score ⇒ score + 5)
4      .ToArray();
5  Console.WriteLine(string.Join(" ", bonusScores));
```

出力

```
75 50 95 65 90
```

POINT　( ) の中は「1 つ分をどう変えるか」。入力の Select(int.Parse) と同じ仲間。

5 行目　Any / All

Any / All：あるか・すべてか（型⑥）

for で書くと

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  bool hasZero = false;
3  for (int i = 0; i < scores.Length; i++)
4  {
5      if (scores[i] == 0)
6      {
7          hasZero = true;
8      }
9  }
10 Console.WriteLine(hasZero);
```

ラムダ式で書くと

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  bool hasZero = scores.Any(score ⇒ score == 0);
3  bool isAllPass = scores.All(score ⇒ score ⩾ 60);
4  Console.WriteLine(hasZero);
5  Console.WriteLine(isAllPass);
```

出力

```
False
False
```

POINT　Any は「1 つでも true があれば true」、All は「すべて true なら true」。型⑥のフラグを 1 行で書ける。

6 行目　OrderBy

OrderBy / OrderByDescending：並べ替え

C# Program.cs

```
1  int[] scores = { 70, 45, 90, 60, 85 };
2  int[] ascending = scores.OrderBy(score ⇒ score).ToArray();
3  int[] ranking = scores.OrderByDescending(score ⇒ score).ToArray();
4  Console.WriteLine(string.Join(" ", ascending));
5  Console.WriteLine(string.Join(" ", ranking));
6  Console.WriteLine(string.Join(" ", scores));
```

出力

```
45 60 70 85 90
90 85 70 60 45
70 45 90 60 85  ← scores は元のまま
```

	Array.Sort	OrderBy
元の配列	書き換わる	そのまま
結果	なし（void）	新しい配列
大きい順	Sort → Reverse	OrderByDescending
並べる基準	値そのもの	=> の右で決める

POINT　score => score は「何を基準に並べるか」。数値の配列なら値そのもの。次のページのクラスの配列で、この部分が活きてくる。

ラムダ式の本領

クラスの配列を、点数の高い順に並べる

C# Program.cs

```
1  class StudentScore
2  {
3      public string Name;
4      public int Score;
5  }
6  class Program
7  {
8      static void Main()
9      {
10         int n = int.Parse(Console.ReadLine());
11         StudentScore[] students = new StudentScore[n];
12         for (int i = 0; i < n; i++)
13         {
14             string[] words = Console.ReadLine().Split(" ");
15             students[i] = new StudentScore();
16             students[i].Name = words[0];
17             students[i].Score = int.Parse(words[1]);
18         }
19         StudentScore[] ranking = students.OrderByDescending(student ⇒ student.Score).ToArra
20         for (int i = 0; i < ranking.Length; i++)
21         {
22             Console.WriteLine(ranking[i].Name + " " + ranking[i].Score);
23         }
24     }
25 }
```

入力例

```
3
ゆうた 72
さくら 95
けん 58
```

出力

```
さくら 95
ゆうた 72
けん 58
```

19 行目を読む

student => student.Score の意味

`student` ⇒ `student.Score`

- student：配列の中の**生徒 1 人分**（StudentScore 1 つ）
- student.Score：その生徒の**点数**
- →「生徒 1 人を受け取って、その点数を返す」＝**点数を基準に並べて**という意味

書き方	並ぶ順
OrderBy(student => student.Score)	点数の低い順
OrderByDescending(student => student.Score)	点数の高い順
OrderBy(student => student.Name)	名前の辞書順

POINT => の右側を変えるだけで、並べる基準を自由に変えられる。Count・Where でも student => student.Score >= 60 のように使える。

一覧

ラムダ式で使う機能と「型」の対応

型（if / for 授業）	ラムダ式	例 → 結果（scores = 70 45 90）
① 全部に同じこと	Select(s => …).ToArray()	Select(s => s + 5) → 75 50 95
② 条件に合うものだけ	Where(s => 条件).ToArray()	Where(s => s >= 60) → 70 90
③ 数える	Count(s => 条件)	Count(s => s >= 60) → 2
④ 合計する	Sum() ／ Sum(s => …)	Sum() → 205
⑤ 最大・最小	Max() / Min()	Max() → 90
⑥ あるか / すべてか	Any(s => 条件) / All(s => 条件)	Any(s => s == 45) → True
並べ替え	OrderBy / OrderByDescending(s => 基準).ToArray()	OrderBy(s => s) → 45 70 90
重複をなくす	Distinct().ToArray()	{1, 1, 2} → {1, 2}

POINT ラムダ式で書けない・分からないときは、いつでも for 版に戻ってよい。結果は同じ。

LESSON 3

3

Paiza の例題に挑戦

目標：for 版とラムダ式版、両方で解けるようになる

例題 STEP 1

配列操作の機能を使う

配列のソート Array.Sort

https://paiza.jp/works/mondai/array_primer/array_primer__array_sort/edit?language_uid=c-sharp

配列の反転 Array.Reverse

https://paiza.jp/works/mondai/array_primer/array_primer__array_reverse/edit?language_uid=c-sharp

配列の最大最小 Max / Min

https://paiza.jp/works/mondai/array_primer/array_primer__array_minmax/edit?language_uid=c-sharp

要素のカウント Count

https://paiza.jp/works/mondai/array_primer/array_primer__elm_count/edit?language_uid=c-sharp

POINT まずは LESSON 1 の機能で解き、終わったら for 版でも書いてみよう。

例題 STEP 2

ラムダ式で条件を書く

ある数以上以下の要素の列挙 1 Where

https://paiza.jp/works/mondai/array_primer/array_primer__array_ave_step1/edit?language_uid=c-sharp

ある数以上以下の要素の列挙 3 Average + Where

https://paiza.jp/works/mondai/array_primer/array_primer__array_ave_step3/edit?language_uid=c-sharp

重複の削除 Distinct + OrderBy

https://paiza.jp/works/mondai/array_primer/array_primer__deduplication/edit?language_uid=c-sharp

辞書順に出力 1 文字列のソート

https://paiza.jp/works/mondai/array_primer/array_primer__string_dictionary_step1/edit?language_uid=c-sharp

POINT Where の結果は「1 行に 1 つ」で出力する問題が多い。for で出力するか、string.Join("\n", …) を使おう。

例題 STEP 3

組になったデータを並べ替える

昇順ソート出力 OrderBy

https://paiza.jp/works/mondai/prob60/sort_asc_2/edit?language_uid=c-sharp

文字と整数の組のソート クラス + OrderBy

https://paiza.jp/works/mondai/prob60/sort_asc_9/edit?language_uid=c-sharp

POINT 「文字と整数の組のソート」は、LESSON 2 の StudentScore の例とほぼ同じ形。クラス名は中身が分かる名前をつけよう。

POINT 早く終わった人は、STEP 1・2 の問題を「for 版」と「ラムダ式版」の両方で書いてみよう。

授業課題（兼 宿題）

スキルチェックに挑戦

- 授業内で取り組み、時間内に終わらなかった問題は宿題とします

C182：ケンケンパ

https://paiza.jp/challenges/920/ready

C157：花のブーケ

https://paiza.jp/challenges/772/ready

C156：勤怠管理

https://paiza.jp/challenges/750/ready

POINT スキルチェックは自分の力で解くこと。困ったら「配列操作 一覧」「ラムダ式と型の対応」の一覧ページと Paiza 辞書を引こう。